The University of Mindanao

IT16/L – Information Assurance and Security 1

8442

(Learnlink: Student Resource Management System)

PROJECT SECURITY DOCUMENTATION HANBOOK

Submitted to:

Renz Arriola

Submitted by:

Jester A. Patriarca

May 2026

1. Project Overview

LearnLink is a web-based Knowledge Management System designed to support educational institutions by providing a centralized platform for storing, managing, and sharing educational resources. They will provide organized and collaborative access to learning materials to enhance their knowledge and learnings.

Role	Access / Purpose
Super Admin	Platform-level system administrator. Has access to all files, can manage global settings, and has a multi-school switcher to view/control all institutions.
Manager	School-level administration (such as Principals or Admin Staff). Can manage users, oversee departments, moderate resources, and handle school-specific settings.
Contributor	Content creators (primarily Teachers). Can create, edit, upload, and publish resources, modules, and review materials.
Student	Learners (Grades 7–10). Can search, view, read, and download learning materials, and receive AI-driven recommendations.

Table 1: **Roles and Access/Purpose**

Layer / Feature	Technology
Backend Framework	ASP.NET Core 8.0 MVC (C#)
Frontend UI	Razor Views (.cshtml), HTML5, CSS3, Bootstrap 5
Database	SQL Server
ORM (Data Access)	Entity Framework Core 8.0
Authentication	ASP.NET Core Identity + Google OAuth 2.0
File Storage	Hybrid: Local System Storage & ASP Cloud Storage/Google Drive API

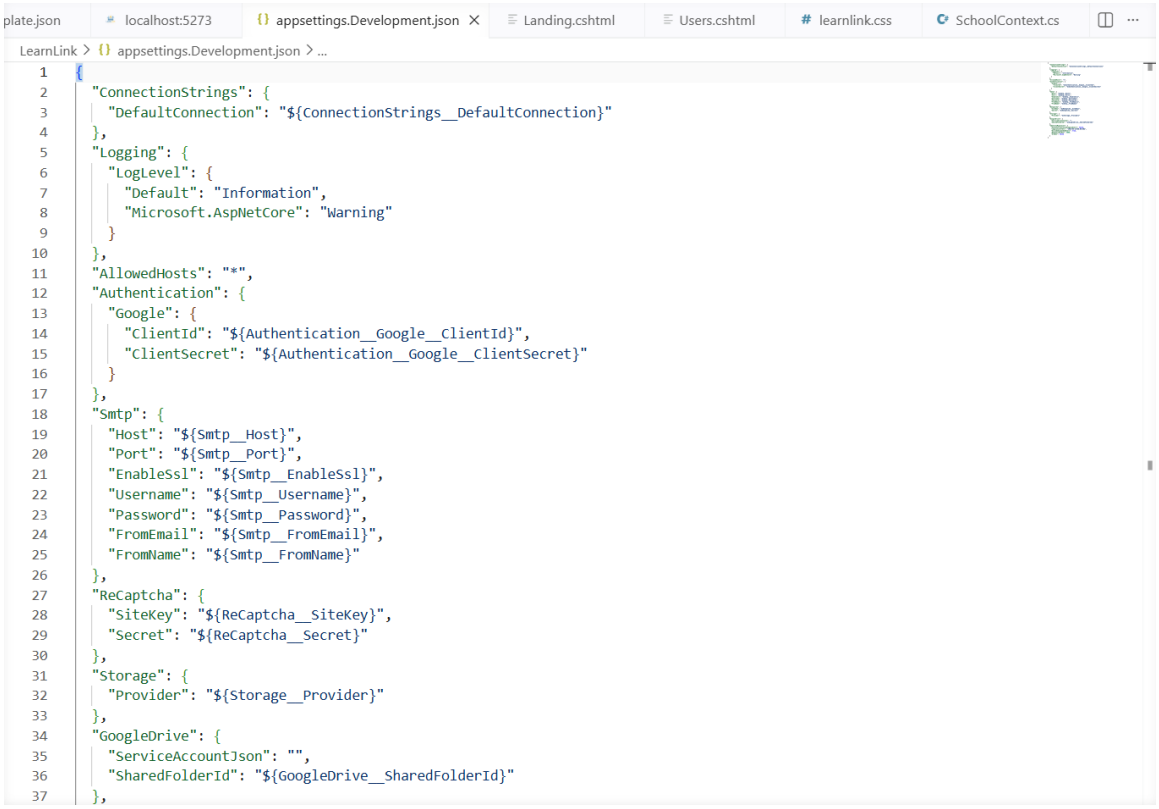
Layer / Feature	Technology
Email Services	Custom SMTP Service
AI / Machine Learning	K-Nearest Neighbors (KNN) Recommendation Engine
Security	Application Cookies (Identity), Google ReCaptcha

Table 2: **Platform/Technology**

2. Secure Coding Practices

1. Appsettings.Development.json (pushed into github and has placeholders)

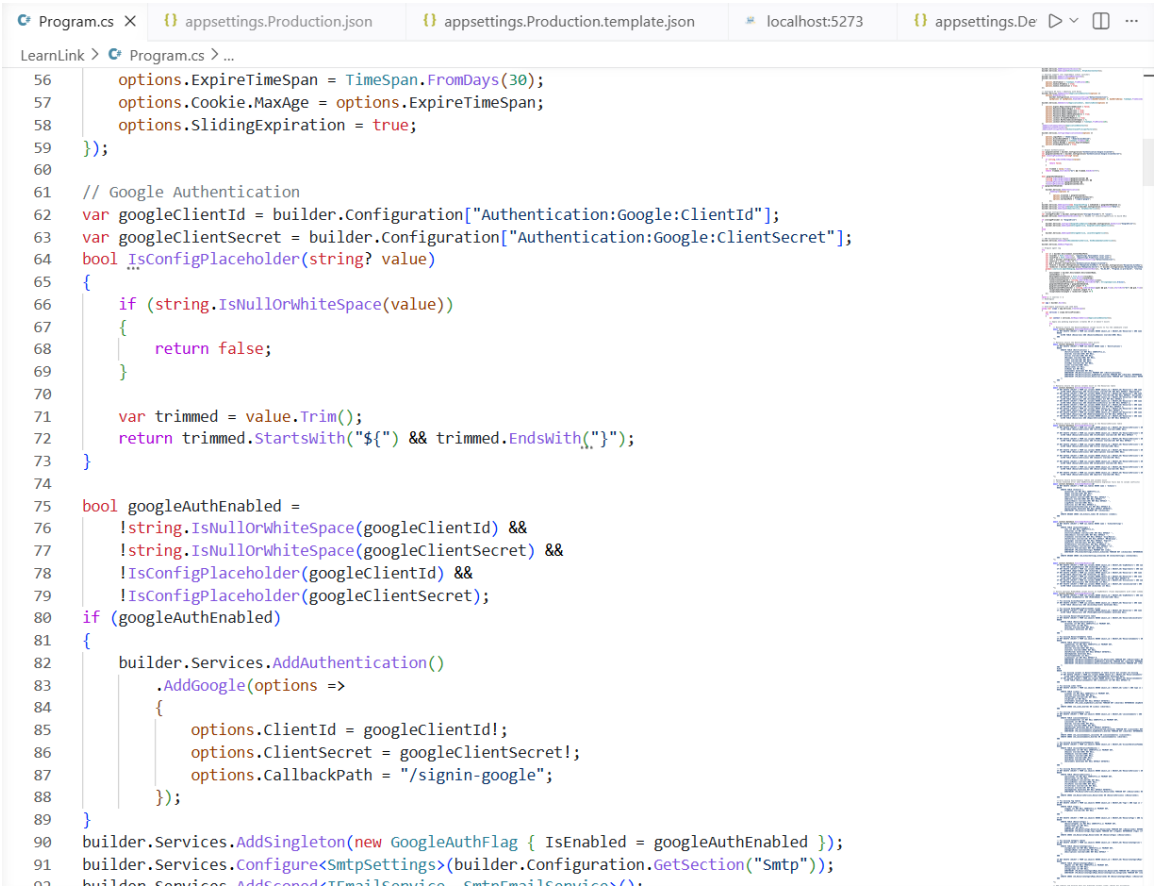
To prevent hardcoded credentials from accidentally entering version control and manipulation when publishing online, these placeholders were placed to prevent it.



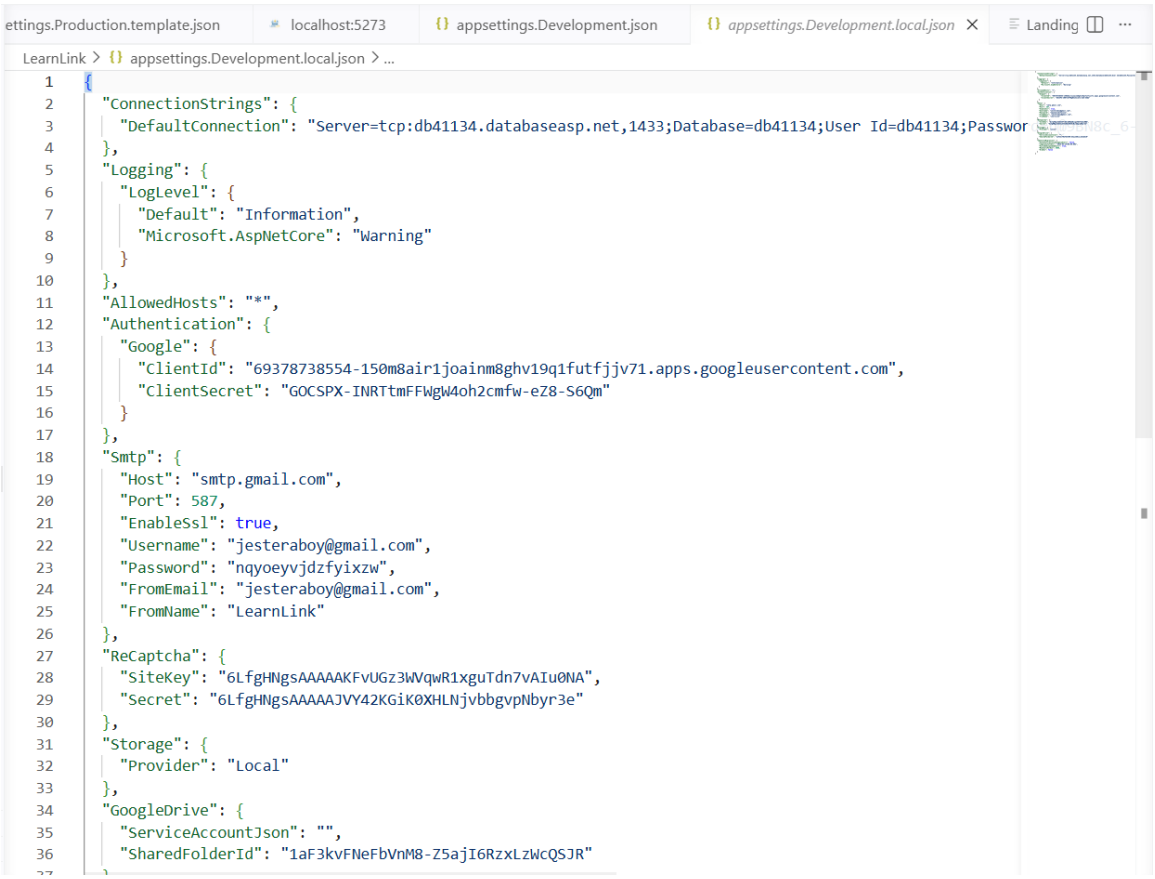
2.GoogleAuth Security (Located in Program.cs)

By checking (line 9 to 12), the application dynamically loads the git-ignored local configuration file. Doing this ensures the application only uses externalized configuration providers rather than reading hard-coded string literals. The (optional: true, reloadOnChange: true) ensures that if the app goes to production, it won't crash when it fails to find the developer's local override file."





This appsettings is being not pushed in github but being placed in both local and asp.net hosting to ensure their connection in both local and cloud services, and to make sure they are working and sync together when there's a change happening in the system.



3. Authentication and Authorization

The system uses Microsoft's ASP.NET Core Identity combined with external OAuth 2.0 (Google) for its core authentication architecture. The system is designed to gracefully handle standard credentials while enforcing strict security measures like account lockout, suspension checks, and anti-bot verification.

Step-by-Step Login & Registration Process

Registration Flow:

1. **Validation:** The user submits their details (Name, Email, Password, School). The server validates the password against strict complexity requirements (min 12 chars, upper/lower/number/special).

2. **Account Creation:** The backend passes the credentials. The plain-text password is hashed and never stored.

3. **Sign-In & Audit:** Upon success, an Audit Log is generated, and the system signs the user in via a secure HTTP-Only Application Cookie.

Login Flow:

1. **Status Check:** Before the users logging into their system, the status is still Inactive (can be seen only in admin & manager roles). The status will change into active if they are logging in the system.

2. **Credential Check:** In the login page, they should input their right email address and password to enter into the system.

3. **Lockout Enforcement:** If the password is wrong, the failed attempt counter increases. If a user fails 7 consecutive times, the account is locked for 15 minutes, and a notification is sent to the SuperAdmin.

4. **reCAPTCHA Challenge:** If the user is logging in from an unrecognized device or triggering suspicious internal flags, they are intercepted by a Post-Auth Google reCAPTCHA screen. Only after the reCAPTCHA token is verified directly against Google's servers does the user get full access.

```
// Program.cs: Password and Lockout Configuration
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequireDigit = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireUppercase = true;
    options.Password.RequireNonAlphanumeric = true;
    options.Password.RequiredLength = 12;
    options.Lockout.AllowedForNewUsers = true;
    options.Lockout.MaxFailedAccessAttempts = 7;
    options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
})
```

Figure 3: Identity configuration enforcing a 12-character minimum with varied character sets, and instituting the 7-attempt lockout threshold.

It utilizes the built-in ASP.NET Core Identity Password Hasher. Identity uses **PBKDF2 with HMAC-SHA256**, a highly secure key derivation function. It automatically generates a unique cryptographic for every single user, combines it with the password, and hashes it iteratively thousands of times. This makes brute-force decoding essentially mathematically impossible.

HomeController.cs | HomeController.cs (Working Tree) | Schools.cshtml | BestPractices.cshtml | Upload.cshtml | ResourceDetail.cshtml | D

LearnLink > Controllers > HomeController.cs > { } LearnLink.Controllers > HomeController > _googleAuthEnabled : bool

```
19  namespace LearnLink.Controllers
21      public class HomeController : Controller
978         public async Task<IActionResult> Login(string email, string password, bool rememberMe)
1022                }
1023
1024                return await FinishPostAuthLoginAsync(user.Id, rememberMe);
1025            }
1026
1027            if (result.IsLockedOut)
1028            {
1029                var lockedOutUser = await _userManager.FindByIdAsync(user.Id);
1030                var lockoutEnd = lockedOutUser?.LockoutEnd;
1031                var remainingMinutes = lockoutEnd.HasValue
1032                    ? Math.Max(1, (int)Math.Ceiling((lockoutEnd.Value.UtcDateTime - DateTime.UtcNow).TotalMinutes))
1033                    : 15;
1034
1035                ViewBag.Error = $"Too many failed login attempts. Please wait {remainingMinutes} minute(s) before trying again.";
1036                ViewBag.ShowForgotPasswordModal = true;
1037                ViewBag.FailedAttempts = 7;
1038                ViewBag.AttemptsRemaining = 0;
1039
1040                await LogAuditAsync("Lockout", "Failure", "Account locked due to multiple failed login attempts", user.Id, email, user.
1041
1042                // Notify admins (SuperAdmin and Manager) about the locked account
1043                try
1044                {
1045                    var admins = new List<ApplicationUser>();
1046                    var superAdmins = await _userManager.GetUsersInRoleAsync("SuperAdmin");
1047                    var managers = await _userManager.GetUsersInRoleAsync("Manager");
1048                    if (superAdmins != null) admins.AddRange(superAdmins);
1049                    if (managers != null) admins.AddRange(managers);
1050
1051                    var recipients = admins.GroupBy(a => a.Id).Select(g => g.First()).ToList();
1052                    foreach (var admin in recipients)
1053                    {
1054                        _context.Notifications.Add(new Notification
1055                        {
```

Figure 4: Lockout interception logic that freezes the account and logs an audit trail to alert administrators.

HomeController.cs | HomeController.cs (Working Tree) | Schools.cshtml | BestPractices.cshtml | Upload.cshtml | ResourceDetail.cshtml | Dashboa

LearnLink > Controllers > HomeController.cs > { } LearnLink.Controllers > HomeController > _googleAuthEnabled : bool

```
19  :e LearnLink.Controllers
21  ic class HomeController : Controller
1126 public async Task<IActionResult> PostAuthRecaptcha(string? flow, string? email)
1156                ViewBag.ReCaptchaSiteKey = _configuration["ReCaptcha:SiteKey"] ?? string.Empty;
1157                ViewBag.PostAuthFlow = postAuthFlow ?? "Login";
1158                ViewBag.Error = "Please complete the reCAPTCHA challenge to continue.";
1159                return View();
1160            }
1161
1162            using var http = new HttpClient();
1163            var values = new Dictionary<string, string>
1164            {
1165                { "secret", recaptchaSecret },
1166                { "response", recaptchaResponse },
1167                { "remoteip", HttpContext.Connection.RemoteIpAddress?.ToString() ?? string.Empty }
1168            };
1169
1170            var resp = await http.PostAsync("https://www.google.com/recaptcha/api/siteverify", new FormUrlEncodedContent(values));
1171            var json = await resp.Content.ReadAsStringAsync();
1172            using var doc = JsonDocument.Parse(json);
1173            if (!doc.RootElement.TryGetProperty("success", out var success) || !success.GetBoolean())
1174            {
1175                _logger?.LogWarning("Post-auth reCAPTCHA verification failed for flow {Flow} from {IP}", postAuthFlow, HttpContext.Connecti
1176                ViewBag.ReCaptchaSiteKey = _configuration["ReCaptcha:SiteKey"] ?? string.Empty;
1177                ViewBag.PostAuthFlow = postAuthFlow ?? "Login";
1178                ViewBag.Error = "reCAPTCHA verification failed. Please try again.";
1179                return View();
1180            }
1181        }
1182    }
1183    catch (Exception ex)
1184    {
1185        _logger?.LogWarning(ex, "Error while verifying post-auth reCAPTCHA");
1186        ViewBag.ReCaptchaSiteKey = _configuration["ReCaptcha:SiteKey"] ?? string.Empty;
1187        ViewBag.PostAuthFlow = postAuthFlow ?? "Login";
1188        ViewBag.Error = "reCAPTCHA verification error. Please try again.";
1189        return View();
```

Figure 5: Backend token verification preventing automated scripts from bypassing the login page.

```
// HomeController.cs: Role-Based Endpoints
[Authorize(Roles = "SuperAdmin,Manager")]
public async Task<IActionResult> ApproveResource(int id)
{
    // Resource approval logic
}

[Authorize(Roles = "Student")]
public async Task<IActionResult> StudentDashboard()
{
    // Student analytics rendering
}

[Authorize(Roles = "SuperAdmin,Contributor,Manager")]
public async Task<IActionResult> UploadResource(IFormFile? file, ...)
{
    // Secure upload logic
}
```

Figure 6: Declarative security attributes ensuring only explicitly elevated roles can approve uploads or access specific dashboards or upload system endpoints.

Utilizing the system role attribute to explicitly whitelist which roles may execute specific backend functions. If a user attempts to navigate to a URL they are not authorized for, the system automatically intercepts the request and routes them to the "/Home/AccessDenied" route.

4. Data Encryption

Data	Method
Passwords	PBKDF2 with HMAC-SHA256 (one-way hash)
SQL connection	Encrypt=True (TLS in-transit)
Auth Cookies	AES-256-CBC (Data Protection API)

Figure 7: Data Encryption

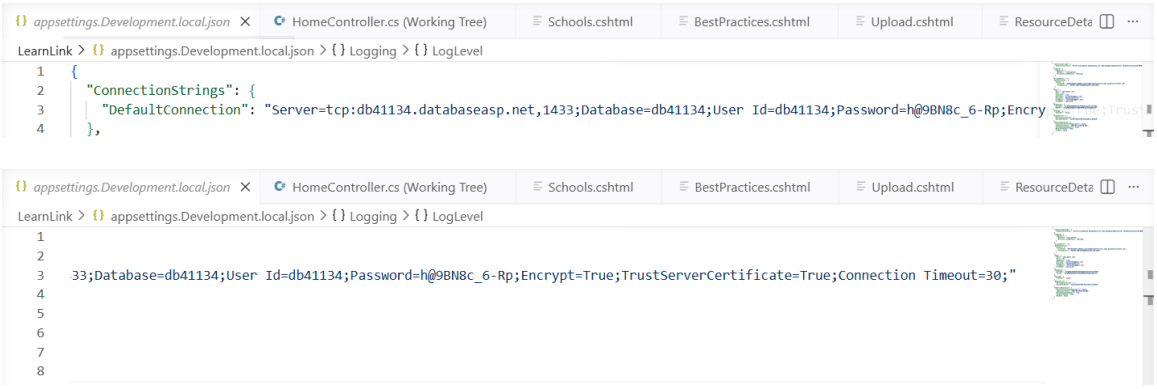
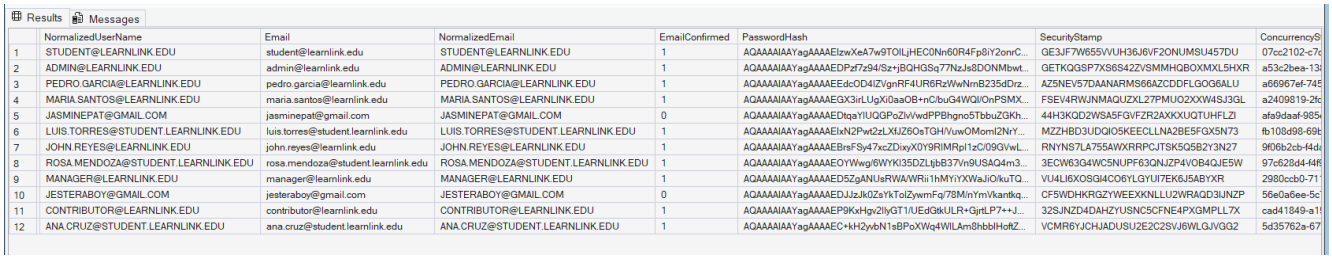


Figure 8: **Encryption in connection string**

```
1015
1016        var result = await _signInManager.CheckPasswordSignInAsync(user, password, lockoutOnFailure: true);
1017        if (result.Succeeded)
1018        {
1019            if (ShouldRequirePostAuthRecaptcha(user))
1020            {
1021                return StartPostAuthRecaptchaFlow("Login", user.Id, rememberMe);
1022            }
1023
1024            return await FinishPostAuthLoginAsync(user.Id, rememberMe);
1025        }
```

Figure 9: **PBKDF2 with HMAC-SHA256 password hashing (Home Controller)**


Results Messages

	NormalizedUserName	Email	NormalizedEmail	EmailConfirmed	PasswordHash	SecurityStamp	ConcurrencyS
1	STUDENT@LEARNLINK.EDU	student@learnlink.edu	STUDENT@LEARNLINK.EDU	1	AQAAAAIAAYagAAAAElzwXeA7w9TOlLjHEC0Nn60R4Fp8iY2onrC...	GE3JF7W655VVUH36J6VF2ONUMSU457DU	07cc2102-c7c
2	ADMIN@LEARNLINK.EDU	admin@learnlink.edu	ADMIN@LEARNLINK.EDU	1	AQAAAAIAAYagAAAAEDPzf7z94/Sz+jBQHGSq77NzJs8DONMbwt...	GETKQGSP7XS6S42ZVSMMHQBOXMXL5HXR	a53c2bea-13
3	PEDRO.GARCIA@LEARNLINK.EDU	pedro.garcia@learnlink.edu	PEDRO.GARCIA@LEARNLINK.EDU	1	AQAAAAIAAYagAAAAEEdcOD4IZVgnRF4UR6RzWwNrnB235dDrz...	AZ5NEV57DAANARMS66AZCDDFLGOG6ALU	a66967ef-745
4	MARIA.SANTOS@LEARNLINK.EDU	maria.santos@learnlink.edu	MARIA.SANTOS@LEARNLINK.EDU	1	AQAAAAIAAYagAAAAEGX3irLUgXi0aaOB+nC/buG4WQI/OnPSMX...	FSEV4RWJNMAQUZXL27PMUO2XXW4SJ3GL	a2409819-2fc
5	JASMINEPAT@GMAIL.COM	jasminepat@gmail.com	JASMINEPAT@GMAIL.COM	0	AQAAAAIAAYagAAAAEDtqaYIUQGPoZlvVwdPPBhgno5TbbuZGKh...	44H3KQD2WSA5FGVFZR2AXKXUQTUHFLZI	afa9daaf-985
6	LUIS.TORRES@STUDENT.LEARNLINK.EDU	luis.torres@student.learnlink.edu	LUIS.TORRES@STUDENT.LEARNLINK.EDU	1	AQAAAAIAAYagAAAAEIxN2Pwt2zLXfJZ6OsTGH/VuwOMomI2NrY...	MZZHBD3UDQIO5KEECLLNA2BE5FGX5N73	fb108d98-69b
7	JOHN.REYES@LEARNLINK.EDU	john.reyes@learnlink.edu	JOHN.REYES@LEARNLINK.EDU	1	AQAAAAIAAYagAAAAEBrsFSjy47xcZDixyX0Y9RIMRpI1zC/09GiVwL...	RNYNS7LA755AWXRRPCJTSK5Q5B2Y3N27	9f06b2cb-f4d
8	ROSA.MENDOZA@STUDENT.LEARNLINK.EDU	rosa.mendoza@student.learnlink.edu	ROSA.MENDOZA@STUDENT.LEARNLINK.EDU	1	AQAAAAIAAYagAAAAEOYWwg/6WYKI35DZLtjbB37Vn9USAQ4m3...	3ECW63G4WC5NUPF63QNJZP4VOB4QJE5W	97c628d4-f4f
9	MANAGER@LEARNLINK.EDU	manager@learnlink.edu	MANAGER@LEARNLINK.EDU	1	AQAAAAIAAYagAAAAED5ZgANUsRWA/WRii1hMYiYXWaJiO/kuTQ...	VU4LI6XOSGI4CO6YLGYUI7EK6J5ABYXR	2980ccb0-71
10	JESTERABOY@GMAIL.COM	jesteraboy@gmail.com	JESTERABOY@GMAIL.COM	0	AQAAAAIAAYagAAAAEDJJzJkOZsYkTolZywmFq/78M/nYmVkantkq...	CF5WDHKRGZYWEEXKNLLU2WRAQD3IJNZP	56e0a6ee-5c
11	CONTRIBUTOR@LEARNLINK.EDU	contributor@learnlink.edu	CONTRIBUTOR@LEARNLINK.EDU	1	AQAAAAIAAYagAAAAEP9KxHgv2IlyGT1/UEdGtkULR+GjrtLP7++J...	32SJNZD4DAHZYUSNC5CFNE4PXGMPLL7X	cad41849-a1
12	ANA.CRUZ@STUDENT.LEARNLINK.EDU	ana.cruz@student.learnlink.edu	ANA.CRUZ@STUDENT.LEARNLINK.EDU	1	AQAAAAIAAYagAAAAEC+kH2ywbN1sBPoXWq4WILAm8hbbIHofkZ...	VCMR6YJCHJADUSU2E2C2SVJ6WLGJVGG2	5d35762a-67

Figure 10: **SSMS Output (AspNetUsers)**

5. Input Validation and Sanitization

1. **Authentication Inputs:** Email addresses, passwords, names (First, Last, Middle)
2. **Resource Metadata:** Title, subject, grade level, description, category, tags
3. **Search & Filter Parameters:** Search queries, subject filters, grade level filters, resource type filters
4. **File Uploads:** File size, file format, file type verification

5. **Text Fields:** Comments, feedback, rejection reasons (maximum length enforcement)

6. **URL Inputs:** External resource links, preview URLs (URI validation and normalization)

````
'Register.cshtml` (Lines 230-455)

**Real-Time Password Strength Validation:**
```javascript
function updatePasswordStrength() {
    const password = document.getElementById('regPassword').value;

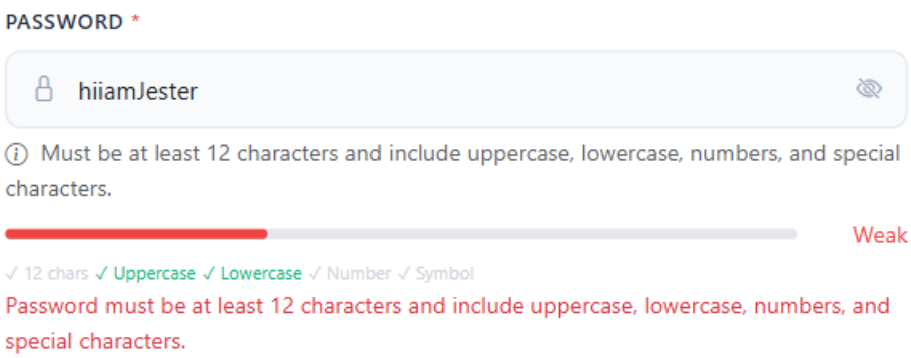
    // Check requirements
    const hasLength = password.length >= 12;
    const hasUpper = /[A-Z]/.test(password);
    const hasLower = /[a-z]/.test(password);
    const hasNumber = /[0-9]/.test(password);
    const hasSymbol = symbolRegex.test(password);  // Special character

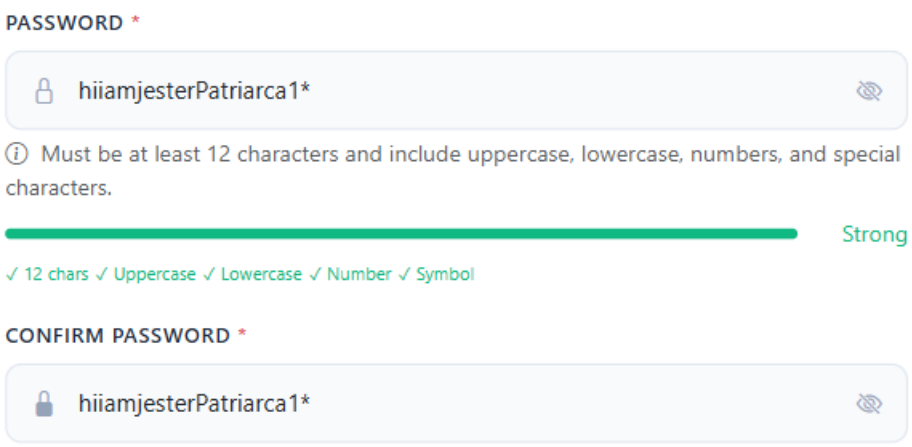
    // Calculate strength (0-5)
    const strength = [hasLength, hasUpper, hasLower, hasNumber, hasSymbol]
        .filter(Boolean).length;

    // Visual feedback
    if (strength <= 2) { /* Red - Weak */ }
    else if (strength === 3) { /* Orange - Medium */ }
    else { /* Green - Strong */ }
}
```
````

Figure 11: Password Strength Validation

Output:





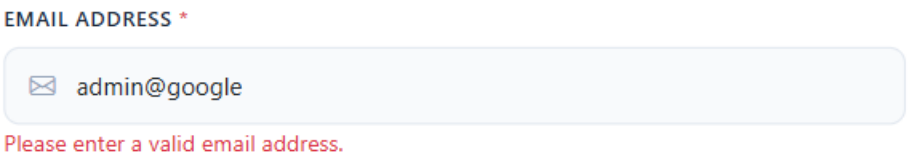
Email Validation:

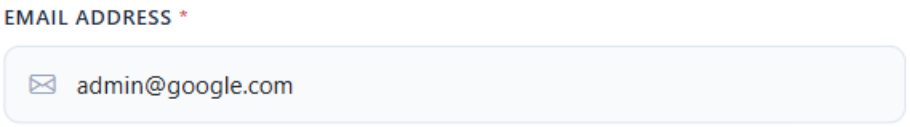
```javascript
function validateEmail() {
    const email = document.getElementById('email').value.trim();
    const emailRegex = /^[^\s@@]+@@[^\s@@]+\.[^\s@@]+$/;

    if (!emailRegex.test(email)) {
        // Show error: "Please enter a valid email address."
        return false;
    }
    return true;
}
```

Figure 12: Email Validation

Output:





Form Submission Prevention:

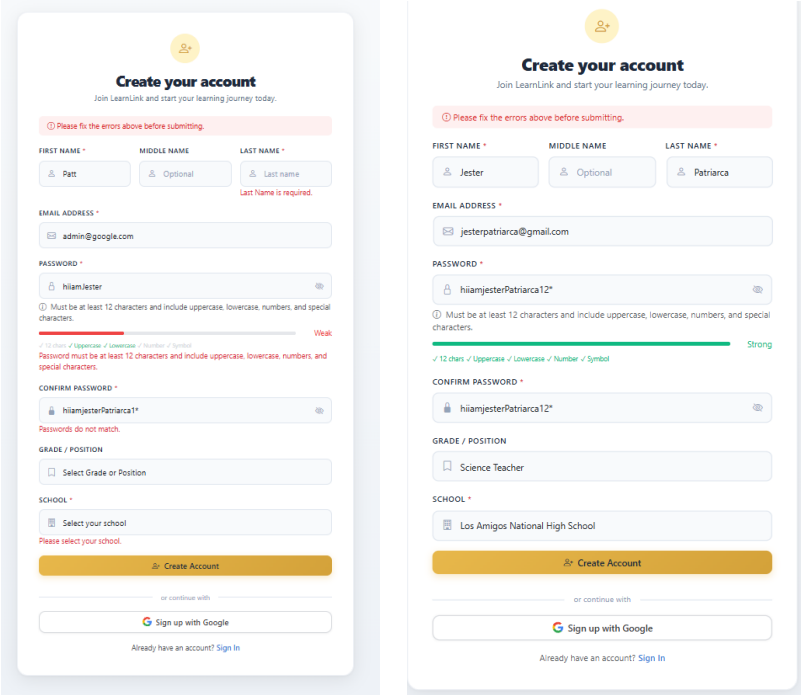
```javascript
document.getElementById('registerForm').addEventListener('submit',
    function(e) {
        e.preventDefault();  // Prevent submission

        // Validate all fields
        const validations = [
            validateFirstName(),
            validateLastName(),
            validateEmail(),
            validatePassword(),
            validateConfirmPassword(),
            validateSchool()
        ];

        // Only proceed if ALL validations pass
        if (validations.every(v => v === true)) {
            this.submit();  // Now safe to submit
        }
    }
);
```

Figure 13: Checking if all credentials are being input and validated.

Output:



6.Error Handling and Logging

HTTP status codes communicate request outcomes to the client. LearnLink uses standardized status codes with corresponding frontend behaviors to provide clear user feedback and security error handling.

Scenario	User Message	Logged Details	Status Code
Database Error	System error. Please try again later.	Full exception + stack trace	500
Not Authorized	You don't have permission for this action.	User ID + attempted action	403
Not Found	The page you requested was not found.	Resource ID + user context	404
Bad Input	Please check your input and try again.	Field name + validation error	400
Account Locked	Your account is locked. Wait 15 minutes.	Lockout duration	401

Figure 14: Error Handling with Status Code

```
11 references
public class AuditLog
{
    [Key]
    0 references
    public int Id { get; set; }

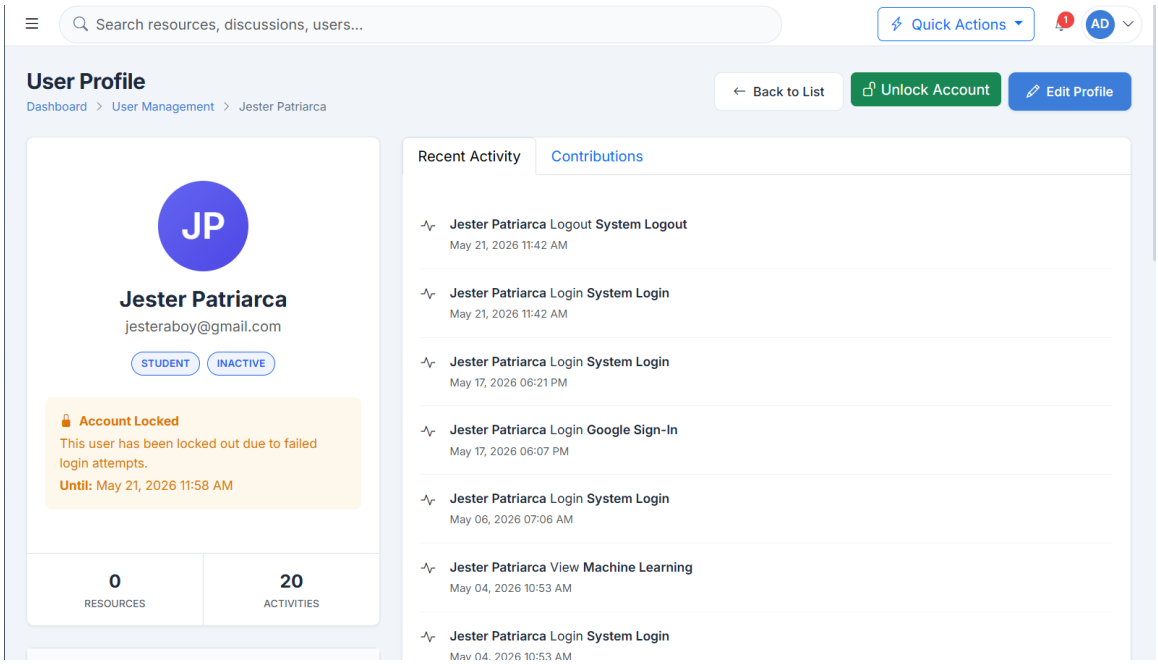
    7 references
    public string? UserId { get; set; }
    [ForeignKey("UserId")]
    14 references
    public ApplicationUser? User { get; set; }

    6 references
    public string? UserEmail { get; set; }

    17 references
    public string Action { get; set; } = "";
    5 references
    public string Status { get; set; } = "Success";
    6 references
    public string? Details { get; set; }
    5 references
    public string? IPAddress { get; set; }
    9 references
    public string? UserAgent { get; set; }
    8 references
    public DateTime Timestamp { get; set; } = DateTime.UtcNow;
    2 references
    public int? SchoolId { get; set; }
}
```

Figure 15: Schemas Audit Logs

Output:



7. Access Control

Sidebar Module	Anonymous	Student	Contributor	Manager	SuperAdmin
Login / Register	✅ Full Access	✅ Full Access	✅ Full Access	✅ Full Access	✅ Full Access
Dashboard	❌ No	❌ Redirects to Repository	❌ Redirects to Repository	✅ Full Access	✅ Full Access
Repository	❌ No	✅ Full Access	✅ Full Access	✅ Full Access	✅ Full Access
Upload Resource	❌ No	❌ No	✅ Full Access	✅ Full Access	✅ Full Access
Knowledge Portal	❌ No	✅ Full Access	✅ Full Access	✅ Full Access	✅ Full Access
Discussions	❌ No	✅ Full Access	✅ Full Access	✅ Full Access	✅ Full Access
Lessons Learned	❌ No	✅ Full Access	✅ Full Access	✅ Full Access	✅ Full Access
Best Practices	❌ No	✅ Full Access	✅ Full Access	✅ Full Access	✅ Full Access
Profile	❌ No	✅ Full Access	✅ Full Access	✅ Full Access	✅ Full Access
Audit Logs	❌ No	❌ No	❌ No	✅ Full Access	✅ Full Access
Backup Management	❌ No	❌ No	❌ No	✅ Full Access	✅ Full Access

Figure 16: **Role-Based Module Access Table**

```
[Authorize(Roles = "SuperAdmin,Manager")]
public async Task<IActionResult> AuditLogs()
{
    // Only SuperAdmin and Manager can access
}
```

```
[Authorize]
public async Task<IActionResult> Repository()
{
    // Any authenticated user (Student, Contributor, Manager, SuperAdmin)
}
```

Figure 17**: Restricted parts in the system that only the allowed roles can access.**

8. Code Auditing Tools

Auditing Tools Used:

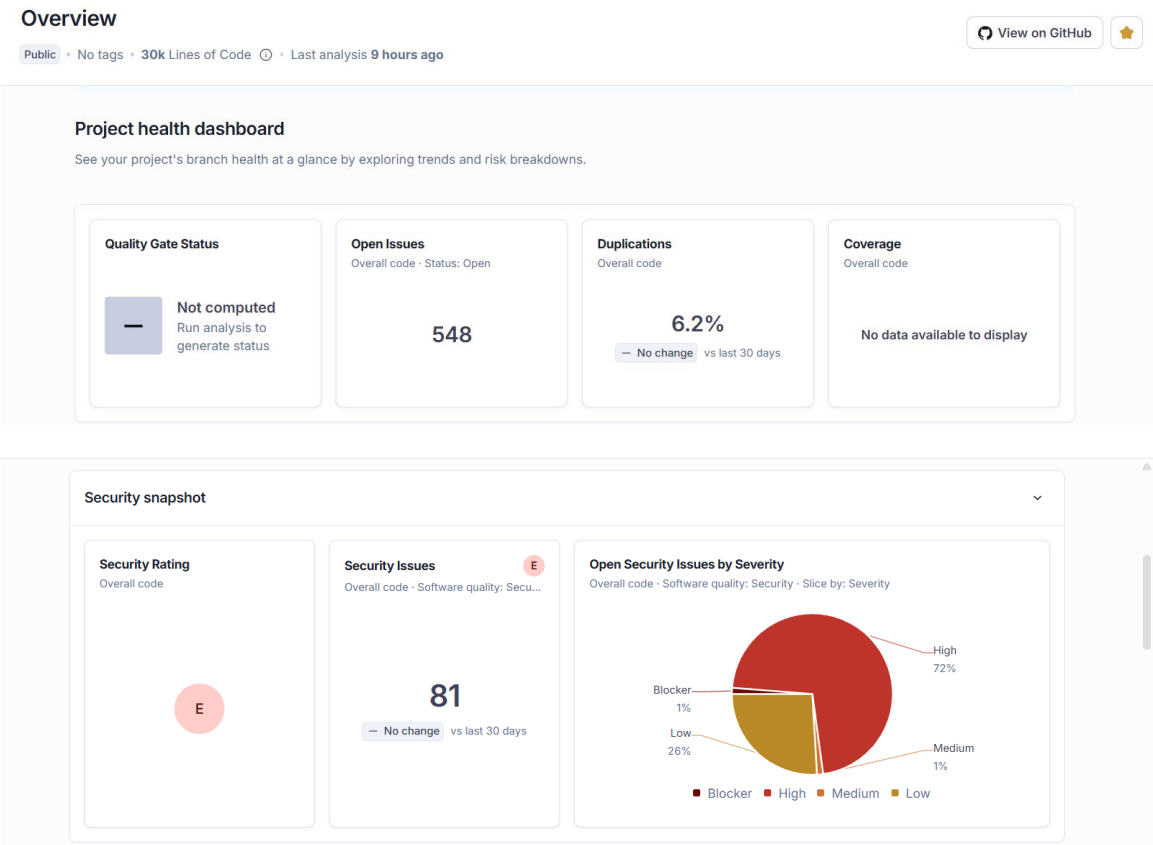
Tool	**What It Does**
Microsoft.CodeAnalysis.NetAnalyzers	Core .NET analyzer pack
SonarCloud	Cloud quality and security analysis
GitHub Actions security workflow	Runs build, dependency, and SAST checks

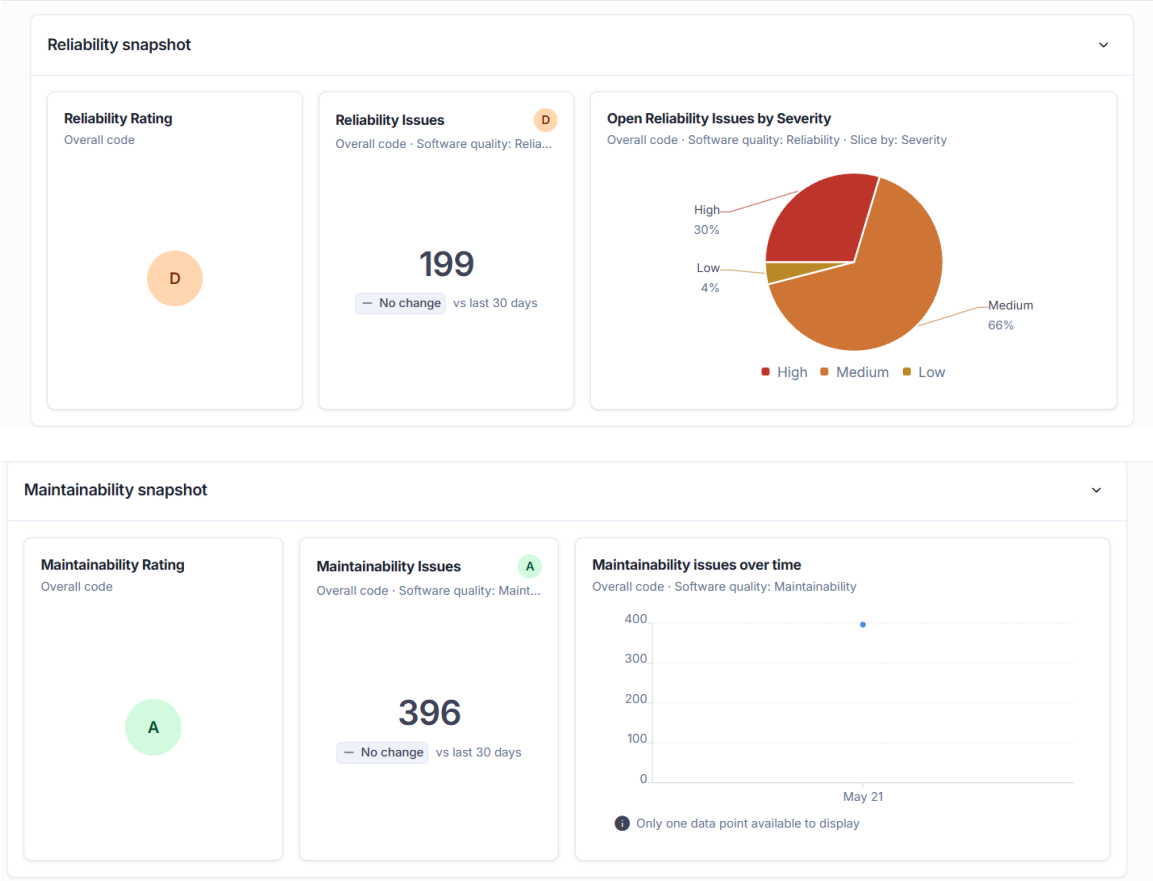
Vulnerabilities Detected and Fixed

Vulnerability	Severity	Detection Method	Status	What Was Fixed
SQL Injection	Critical	CA2100, SecurityCodeScan	Fixed	Parameterized EF/LINQ queries
Hardcoded Secrets	Critical	SecurityCodeScan, workflow review	Fixed	Moved config to secrets / secure config
Weak Cryptography	Critical	CA5350, CA5373	Fixed	Stronger approved crypto / Identity defaults
Missing CSRF Tokens	High	CA5391, code review	Partially Fixed	Added antiforgery to audit admin AJAX; other POST actions still need coverage
Insecure Deserialization	High	CA5394	Fixed	Avoided unsafe deserializers

Certificate Validation	High	CA5359	Fixed	Enforced safer certificate handling
Weak Password Policy	Medium	Code review, tests	Fixed	12-character policy and lockout rules
Missing Auth Checks	Critical	SecurityCodeScan, code review	Fixed	Added/verified authorization gates
Insufficient Logging	Medium	Design review	Fixed / Improved	Audit logging added and expanded
Insecure SSL/TLS	High	CA5390, CA5397	Fixed	Modern TLS configuration

SonarCloud Project Health Dashboard:





9. Testing

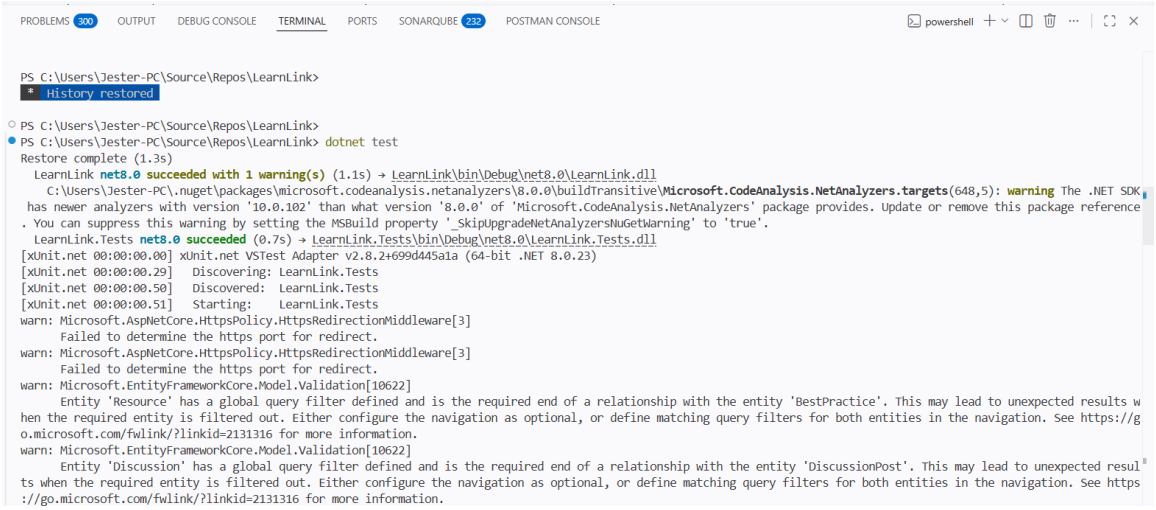
To maintain absolute simplicity and leverage direct environment integration, the test suite relies exclusively on the **Dotnet CLI (dotnet test)** framework

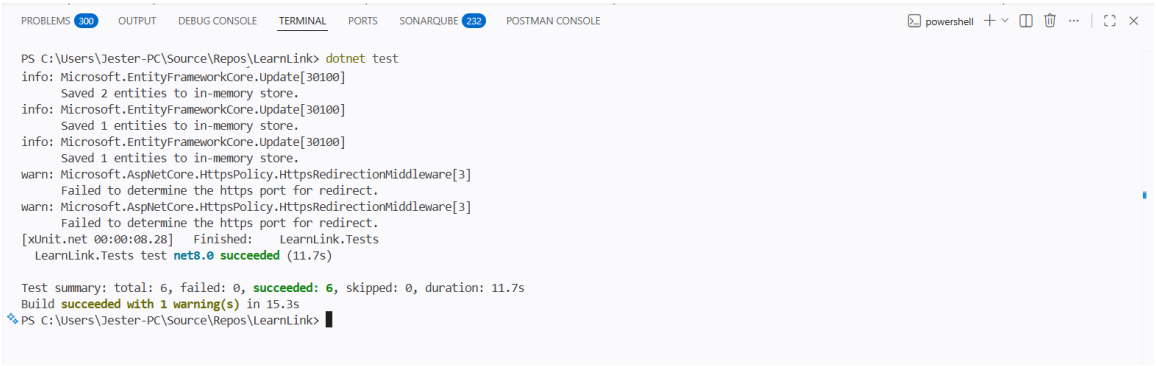
1. **The Dotnet CLI and Test SDK:** The execution runtime is driven entirely by the dotnet test command. This native tool handles dependency resolution, compiles the testing project alongside the main application, runs the selected runner, and outputs standardized test pass/fail results directly to the command line.

2. **xUnit Test Framework:** The integration tests are built upon xUnit, a modern, extensible testing framework for .NET. Using attributes assertion models (such as Assert.Equal and Assert.Contains), xUnit programmatically verifies the states of HTTP responses and view data.

3. **Microsoft.AspNetCore.Mvc.Testing:** Rather than launching a real web server bound to local ports (which is slow and error-prone), This launches an in-memory hosting environment representing the actual application pipeline. It handles internal HTTP routing, middleware activation, and session cookie state tracking.

Doing the tests using dotnet test:





Test Cases Results:

Test ID	Test Method Name	Description / Scope	Expected Result	Status
TC-01	`LoginGet_ReturnsLoginView`	Accesses the login page via GET.	Page returns `HTTP 200 OK` and renders login fields.	**PASSED**
TC-02	`LoginPost_WithMissingCredentials_ShowsValidationMessage`	Submits empty login form via POST.	Returns `HTTP 200 OK` with validation message: *"Please enter email and password."*	**PASSED**
TC-03	`RegisterGet_ShowsActiveSchools`	Accesses the registration page via GET.	Returns `HTTP 200 OK` showing list of active schools.	**PASSED**
TC-04	`RegisterPost_WithMissingSchool_ShowsErrorMessage`	Submits registration form with no school selected.	Returns `HTTP 200 OK` and prompts: *"Please select your school."*	**PASSED**
TC-05	`RegisterPost_WithVerifiedData_RedirectsToRepository`	Submits valid registration details with email pre-verified.	Creates user account, logs in, and redirects (`HTTP 302 Found`) to the resource **Repository**.	**PASSED**
TC-06	`UsersGet_WhenUnauthenticatedRedirectsToLogin`	Unauthorized user attempts to view the user list page.	Correctly redirects (`HTTP 302 Redirect`) to the **Login** route.	**PASSED**

10. Security Policies

- **Password Policy:**
    - Minimum length: 12 characters
    - Must contain at least one uppercase letter (A-Z)
    - Must contain at least one lowercase letter (a-z)
    - Must contain at least one number (0-9)
    - Must contain at least one special character (e.g. !@#$%^&*)
    - Passwords are hashed by ASP.NET Core Identity (PBKDF2 by default) before storage (one-way, salted, iterated — default iterations documented in code).
    - Passwords are never stored or logged in plaintext.
    - Password reset tokens: expire after 1 hour and are single-use.
- **Login Attempt Policy:**
    - Maximum failed login attempts before lockout: 7
    - Lockout duration: 15 minutes before the account is fully back to login
    - Failed authentication attempts are recorded in the audit log with user along with the date and time.
- **Data Handling Policy:**
    - Passwords: one-way hashed (Identity PBKDF2), never retrievable.
    - JWT/session tokens: stored client-side according to your front-end policy

      It stores session authentication data client-side in an **HTTP-only, secure cookie**. This is managed via ASP.NET Core Identity authentication cookies and session state cookie

```
// Session support (for SuperAdmin school switcher)
builder.Services.AddDistributedMemoryCache();
builder.Services.AddMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(60);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});
```

      // Prevents client-side scripts from reading the cookie (mitigates XSS token hijacking)

```
builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Home/Login";
    options.AccessDeniedPath = "/Home/AccessDenied";
    options.ExpireTimeSpan = TimeSpan.FromDays(30);
    options.Cookie.MaxAge = options.ExpireTimeSpan;
    options.SlidingExpiration = true;
});
```

Figure 17 : Authentication Cookie Properties

- Sensitive configuration: kept out of source control (environment variables, Key Vault, GitHub Secrets).
- API responses: ownership/authorization checked before returning user data (prevents BOLA).

  BOLA (**Broken Object Level Authorization**) is prevented by performing explicit ownership checks (r.UserId == currentUser.Id) or permission checks before modifying or returning any object data requested by a client.

```
// Only SuperAdmin can delete anyone's resources.
if (!User.IsInRole("SuperAdmin"))
{
    // Restricts the deletion to ONLY items uploaded by the authenticated user's ID
    resourcesToDelete = resourcesToDelete.Where(r => r.UserId ==
currentUser.Id).ToList();
}
```

Figure 18: Deleting Resource Records

When a client requests to delete one or multiple resource IDs, the application intercepts the query and filters it to only match the currently logged-in user unless they have SuperAdmin privileges:

- Database: SQL Server via Entity Framework Core (parameterized queries by design).
- Backups: encrypted, access controlled, and periodically restored to validate recovery.

- **Audit & Retention:**
  - All security events (login success/failure, lockouts, role changes, password resets, account suspensions) are written to the recent activities in the User's Profile.
  - Access to full audit data restricted to SuperAdmin/Manager roles. UI redacts or truncates sensitive details for non-admins.

- **Operational Notes:**
    - Secrets: store SONAR_TOKEN, cloud storage API keys, SMTP credentials in GitHub Secrets or Key Vault.
    - Make code enforcement the source of truth: password and lockout settings are set in Program.cs (Identity options). If you change policy, update Program.cs and this document.

11. Incident Response Plan

- **Detection:**
    - System administrators and automated logging systems (e.g., Application Insights or built-in error loggers) detect anomalies, such as unauthorized access attempts, sudden spikes in failed logins, or database connection failures.
    - Users report system bugs or missing resources via the support contact channels.
- **Reporting:**
    - Once an incident is detected, an automated alert or manual report is sent to the system administrator.
    - The incident is logged in the internal tracking system detailing the timestamp, affected module (e.g., Knowledge Repository, User Authentication), and severity level.
- **Containment:**
    - If a breach or severe data corruption occurs, the affected user accounts are temporarily suspended to prevent further unauthorized access.
    - The affected module (e.g., file uploads) is being deleted by the admin to prevent security issues in the system.
- **Recovery:**
    - Administrators navigate to the **System Backup & Recovery** dashboard.
    - If a single file was maliciously deleted, the **Archive Management** tab is used to restore the soft-deleted file.
    - If the database was corrupted, the system is rolled back using a previous snapshot from the **Restore History** tab. Passwords are reset for compromised accounts, and the system is brought back online.

RUBRIC – Technical Writing

Criteria	Excellent (90–95)	Very Good (85–89)	Good (80–84)	Poor (75–79)	Needs Improvement (0)
1. Project Overview & Introduction	Clear, complete, and professional overview of project purpose, users, platform, and technologies used.	Overview is complete with minor missing details.	Basic overview provided but lacks depth or clarity.	Incomplete overview with major missing information.	No meaningful project overview provided.
2. Secure Coding Documentation	Thorough explanation of secure coding practices with relevant code samples and screenshots.	Good explanation with minor missing examples/details.	Basic explanation with limited supporting evidence.	Weak explanation with insufficient examples/screenshots.	Missing or unacceptable secure coding documentation.
3. Authentication & Authorization Documentation	Clearly explains authentication flow, hashing methods, roles, and restrictions with strong evidence.	Good explanation with minor omissions.	Basic explanation but lacks detail or evidence.	Incomplete/unclear explanation of authentication/authorization.	Missing documentation for authentication/authorization.
4. Data Encryption Documentation	Fully explains encryption methods, implementation, and encrypted data proof/screenshots.	Good explanation with minor missing details.	Basic encryption explanation with limited evidence.	Weak encryption documentation.	Missing or inaccurate encryption documentation.
5. Input Validation Documentation	Comprehensive explanation of validation/sanitization methods with screenshots/examples.	Good documentation with minor missing details.	Basic documentation with limited examples.	Weak/incomplete validation documentation.	Missing validation documentation.
6. Error Handling & Logging Documentation	Clearly documents error handling workflow and logging strategy with proof/screenshots.	Good explanation with minor omissions.	Basic documentation with limited detail.	Weak/incomplete documentation.	Missing error handling/logging documentation.
7. Testing & Auditing Documentation	Complete documentation of testing procedures, audit tools, vulnerabilities found, and results.	Strong documentation with minor missing evidence.	Basic documentation with partial results/screenshots.	Limited testing/auditing documentation.	Missing testing/auditing documentation.
8. Security Policies & Compliance Handbook	Policies are comprehensive, professional, and aligned with system implementation.	Strong policies with minor missing components.	Basic policies present but lacking detail.	Weak/incomplete policies.	Missing security policies/compliance section.
9. Incident Response Plan	Detailed and realistic incident response plan covering detection to recovery.	Good plan with minor missing steps/details.	Basic response plan provided.	Weak/incomplete response plan.	No incident response plan submitted.
10. Writing Quality & Technical Presentation	Professional formatting, excellent grammar, organized structure, technical clarity, and proper terminology throughout.	Well-written with minor grammar/formatting issues.	Understandable but contains several writing/formatting issues.	Poor organization or frequent writing errors.	Unprofessional, disorganized, or unreadable documentation.